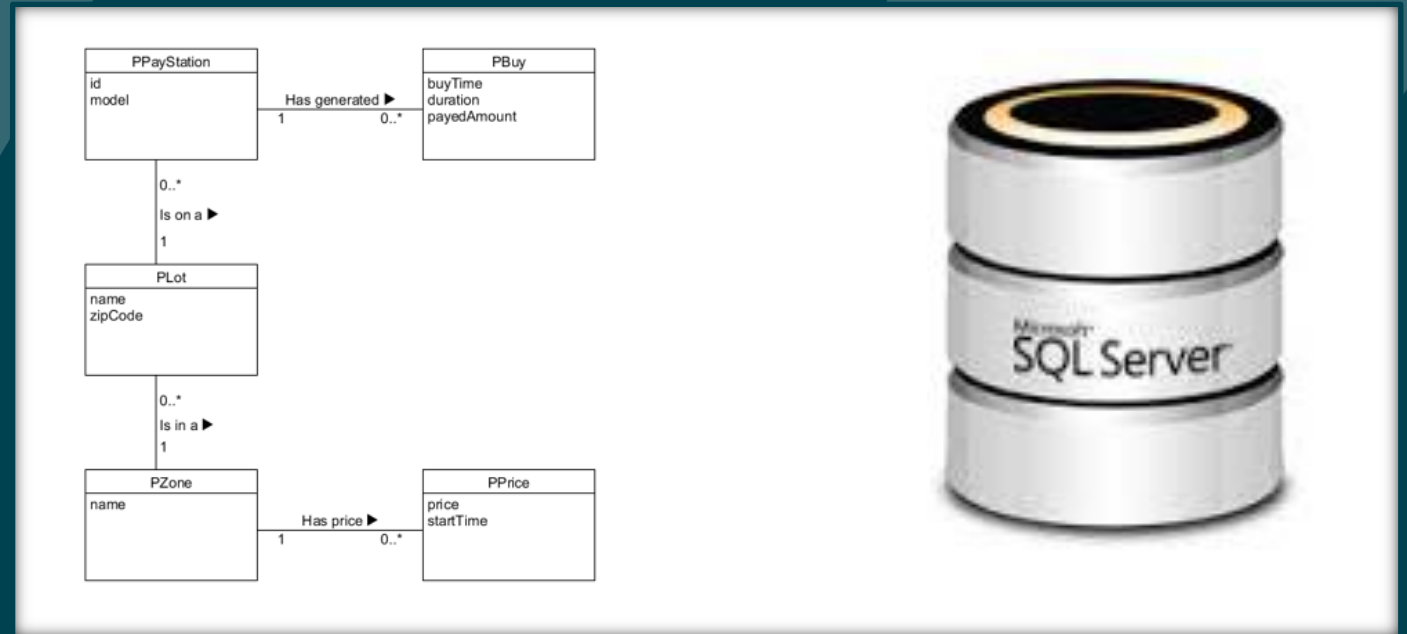


Data Modelling

From domain model to implemented database





- Learning goals
- Domain Model recap
- Data Modeling - Introduction
- From Domain Model to Database
 - How to map relations
 - How to deal with multivalued and composite attributes
 - How to model inheritance in a DB

Learning Goals



PROFESSIONSHØJSKOLEN

Learning Goals

- Be able to **transform a domain model to a relational model** using the transformation rules (*model to model*)
- Understand the process of **turning a conceptual data model into an implemented relational database** (*model to actual DB*)

Domain Model

Recap



PROFESSIONSHØJSKOLEN

Domain Model in UP

s = start, r = refine

Discipline	Artifact	Incep.	Elab.	Const.	Trans.
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	
Implementation	Implementation Model (code,...)		s	r	r

Overview

Preliminary Study

Functional requirements

Workflow
(UML – activity diagram)



Employee-Goal Table

Employee	Task	Goal	Step in task
Shop assistant	Receive new order	Register order	•Find items wanted •Add them to order •Add customer info •Finish order •Order confirmation to customer, warehouse and accounting
.....			

Mock up: Register order

No	Description	Unit price	Quantity	Subtotal
1234	Hammer	100	1	100
1342	Nails	20	4	80

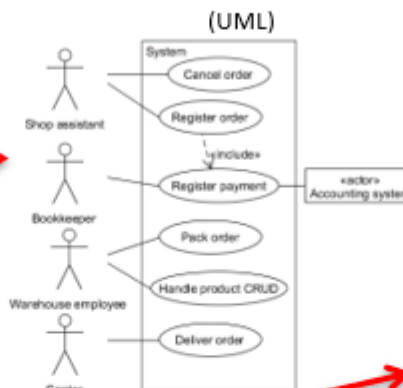
Search: OK Total: 180

Customer
 Name: John Smith
 Address: Hælsundvej 35, 9090 Aalborg
 Phone: 12345678

Order
 No: 3214
 Date: 2013-09-13
 Status: Ordered

Create Print

Use case diagram
(UML)



Brief Use Case (all)

Focus: What the system should do

Use case: Register Order

A customer calls to place an order. The shop assistant uses the system to create a new order, add products, track customer information, save the order, print the order form and invoice. Along the way, the system displays details about the products, subtotal and total

Alternative flow (Causal)

Fully Dressed Use Cases (the most complex)

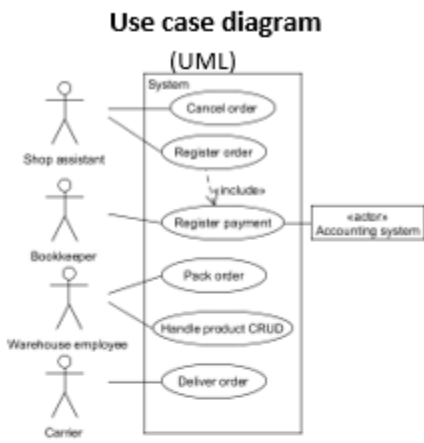
Avoid UI design (mock up) details

Use case name	Register order	
Actors	Shop assistant	
Pre-conditions	The desired products are registered and in stock	
Post-conditions	The order is created and added products or customer information.	
Frequency	100 times a day	
Main Success Scenario (Flow of events)	Actor (Action)	System (Response)
	1. A customer calls to place an order	
	2. The shop assistant begins a new order	3. The system creates a new order
	4. The shop assistant gives the ID of the desired product	5. The system returns the product data and subtotal
	6.	
Alternative flows	4a. The id does not exist. 4b. <u> </u> 6a. <u> </u>	

Overview

Functional requirements

Information requirements



Brief Use Case (all)
Focus: What the system should do

Use case: Register Order
A customer calls to place an order. The shop assistant uses the system to create a new order, add products, track customer information, save the order, print the order form and invoice. Along the way, the system displays details about the products, subtotal and total

Alternative flow (Causal)

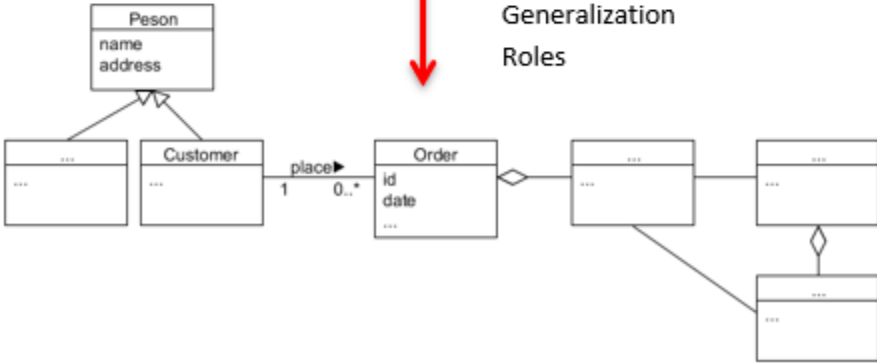
Find candidates for classes: Find Nouns
Select: Are there attributes or events on the class?
Is there more than one object of the class?
Is the class from the problem domain (not a print out)?
Can all events be registered?
Are all the use cases covered?

Candidates for classes	Evaluation	IN/OUT
Customer	Need to register customer information	Included
Employee	Not needed to register who has served the customer	Excluded
.....		

Fully Dressed Use Cases (the most complex)
Avoid UI design (mock up) details

Use case name	Register order	
Actors	Shop assistant	
Pre-conditions	The desired products are registered and in stock	
Post-conditions	The order created and added products or customer information.	
Frequency	100 times a day	
Main Success Scenario (Flow of events)	Actor (Action)	System (Response)
	1. A customer calls to place an order	
	2. The shop assistant begins a new order	3. The system creates a new order
	4. The shop assistant gives the ID of the desired product	5. The system returns the product data and subtotal
	6.	
Alternative flows	4a. The id does not exist. 4b. 6a.	

Domain Model

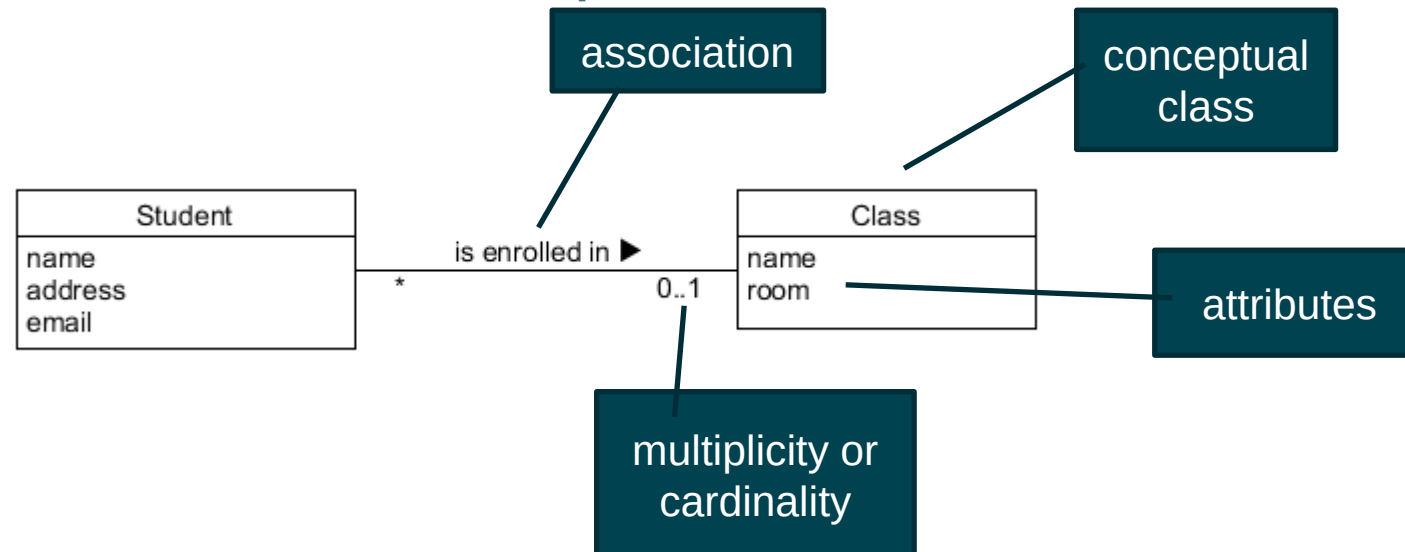


Consider:
Association
Aggregation
Generalization
Roles

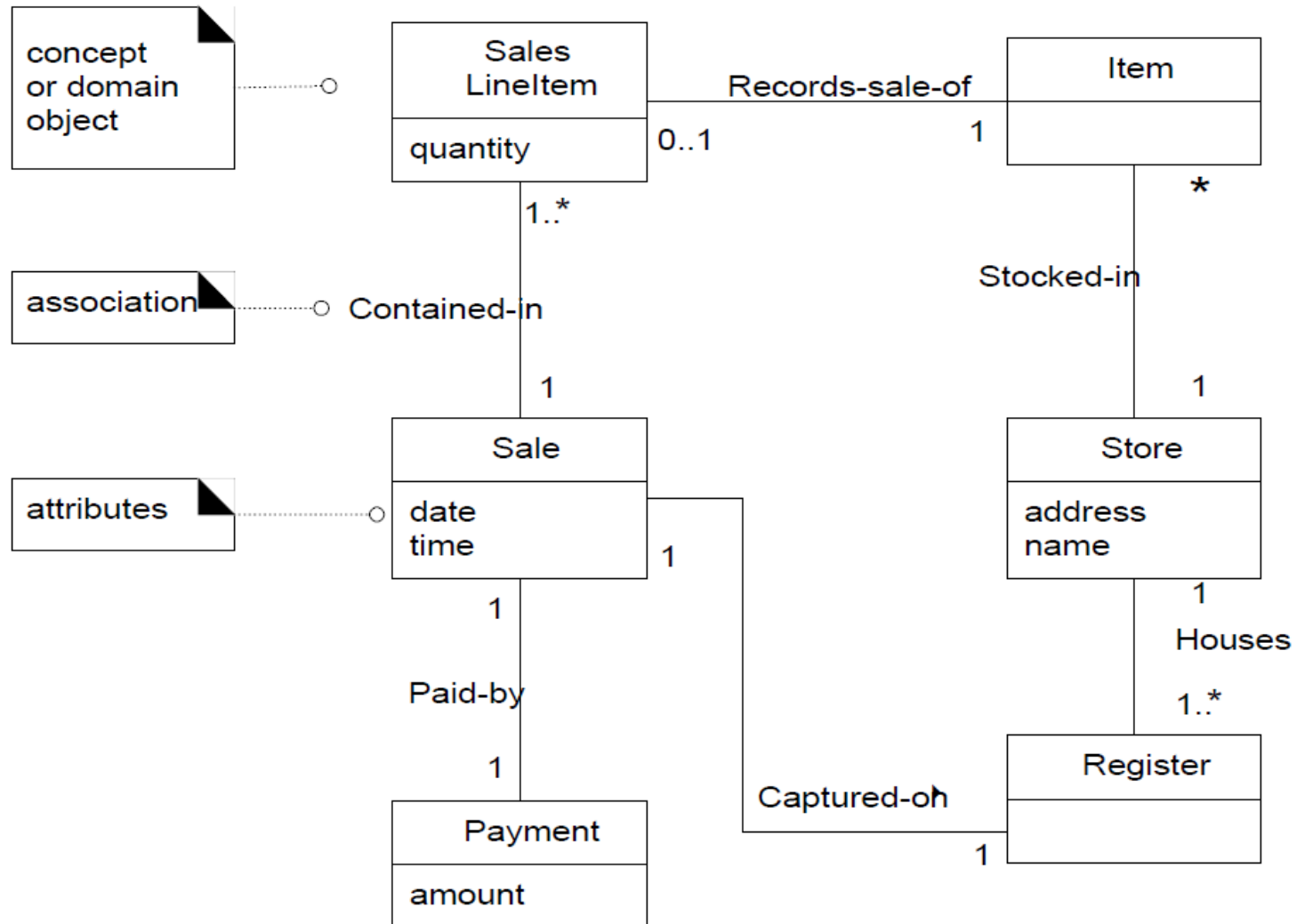
Domain Model

- *Visual* representation of conceptual classes or real-world objects in a domain of interest
- **Domain model** represents:
 - **concepts** of the problem domain (mini-world)
 - **attributes** which describes the conceptual classes
 - **associations** between conceptual classes

- Example:



Domain Model Example



Conceptual Class

An abstraction of persons, things, agreements concepts in the Problem Domain
(real world objects/concepts)

Example:

Problem Domain
(real world)



← Customer

Domain Model
(model of the
Problem Domain)

Customer
name phone email

Conceptual Class
Customer

The Conceptual Class **Customer** is used to describe
real-world objects:

Example:

Hans Hansen, phone: 98123456,
email: hh@gmail.com

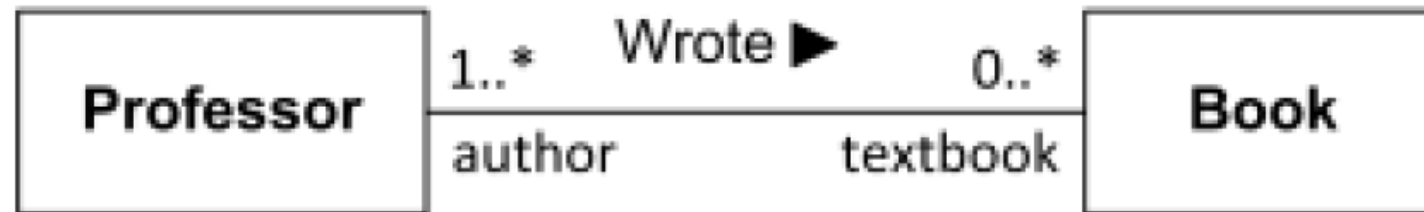
<u>Hans Hansen: Customer</u>
name = Hans Hansen phone = 98123456 email = hh@gmail.com

Object
Customer
(not sw object
at this point)

Domain Model

Associations - Multiplicity

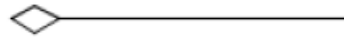
- Associations define how conceptual classes are related (or associated)
- Example:



Domain Model - Association Types



generalization



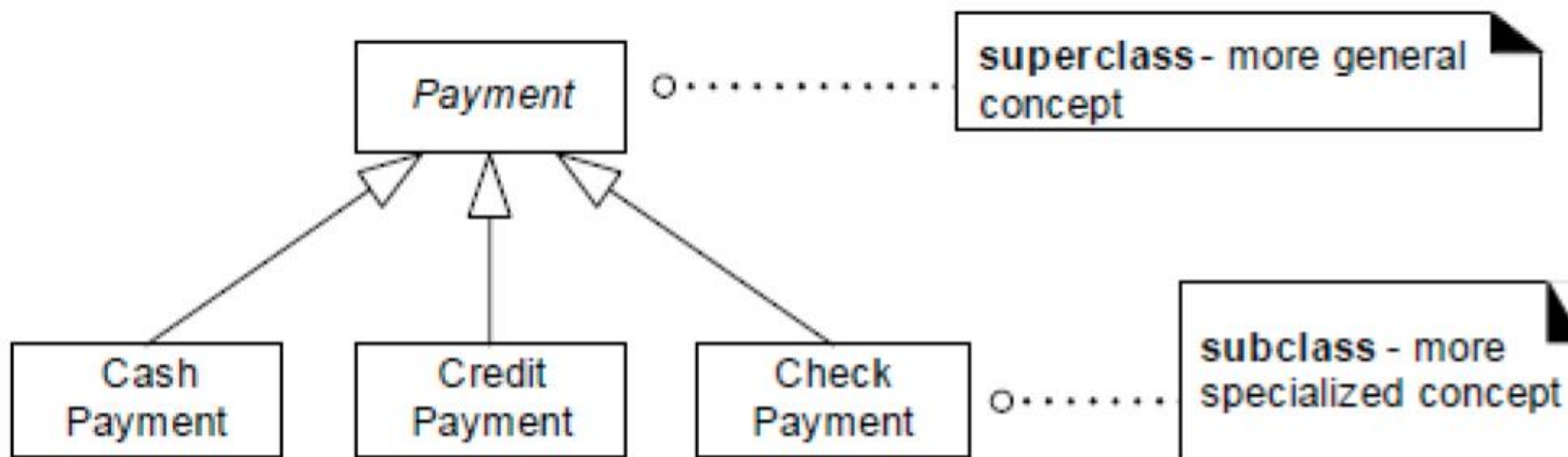
aggregation



composition

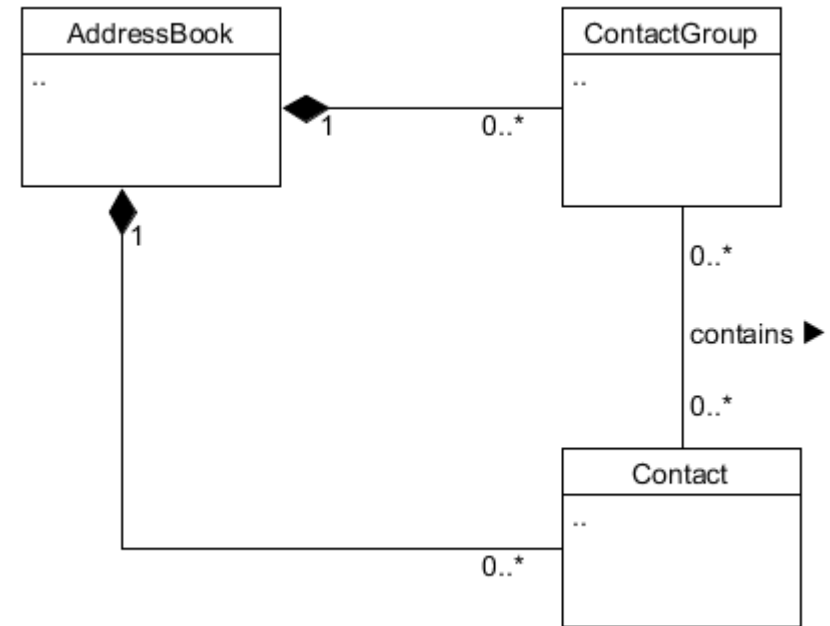
in UML is an aggregation
with black background

Generalization ("...is a...")



Composition (composite aggregation)

- Strong type of aggregation (hand – finger)
- If the composite part of a composition is deleted, its parts must either be destroyed, or attached to another composite (e.g. order – orderline). No free-floating fingers allowed!



Aggregation

- The aggregation loosely suggests whole-part relationships (as do many ordinary associations).
- Larman quote: *“It has no meaningful distinct semantics in UML versus a plain association”*
- Rumbaugh quote: *“In spite of a few semantics attached to aggregation, everybody thinks it is necessary (for different reasons). Think of it as a modelling placebo”*

Data Modelling

Intro



PROFESSIONSHØJSKOLEN

Database Design Process

1. Make Domain Model (conceptual model)

2. Map Domain Model to Relational Model

- Convert Domain model to Relational Database Schema based on simple rules (this module)

3. Normalize Relational Model

- Optimize data model for Database (next module)

4. Implement databases and tables (etc.)

What mapping from OOP to relational DB feels like

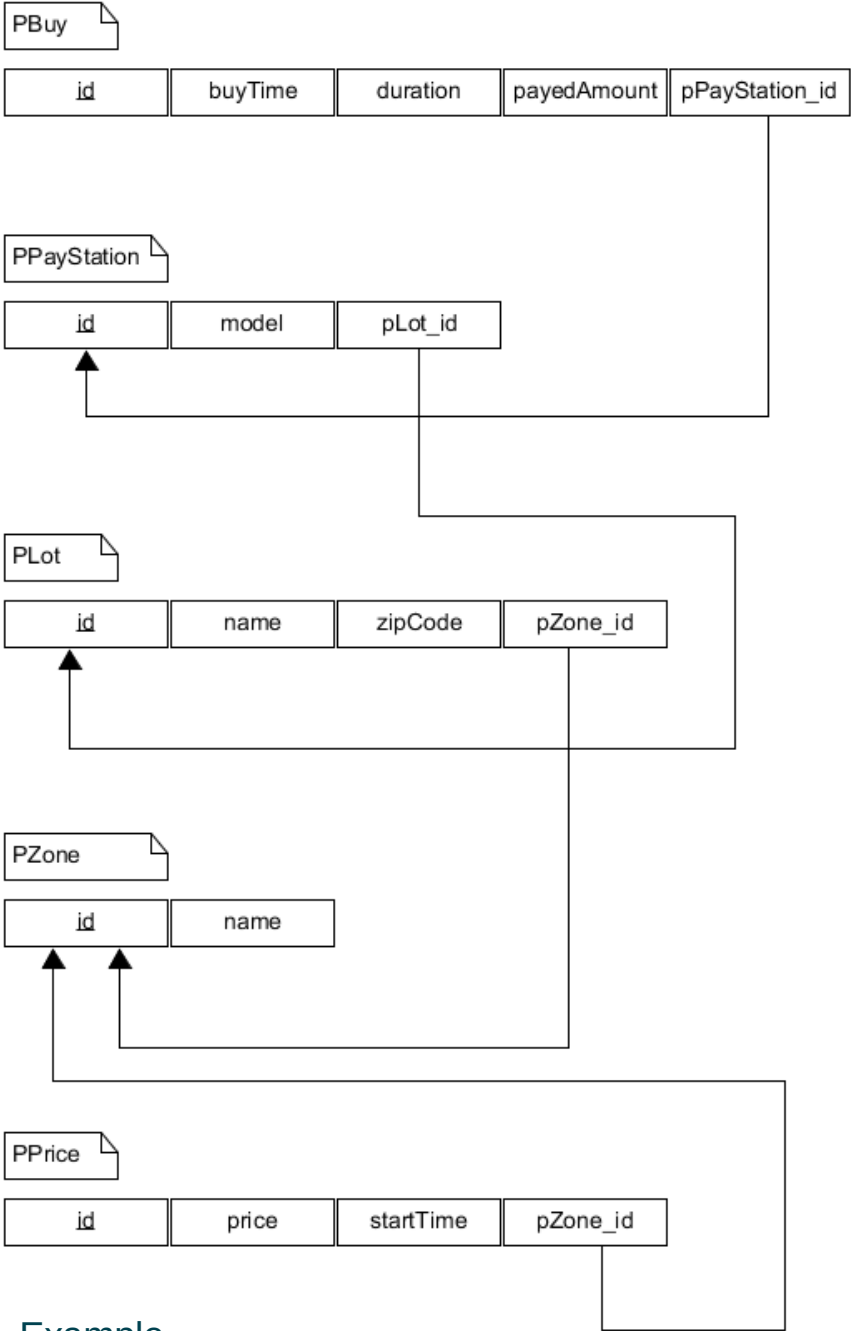
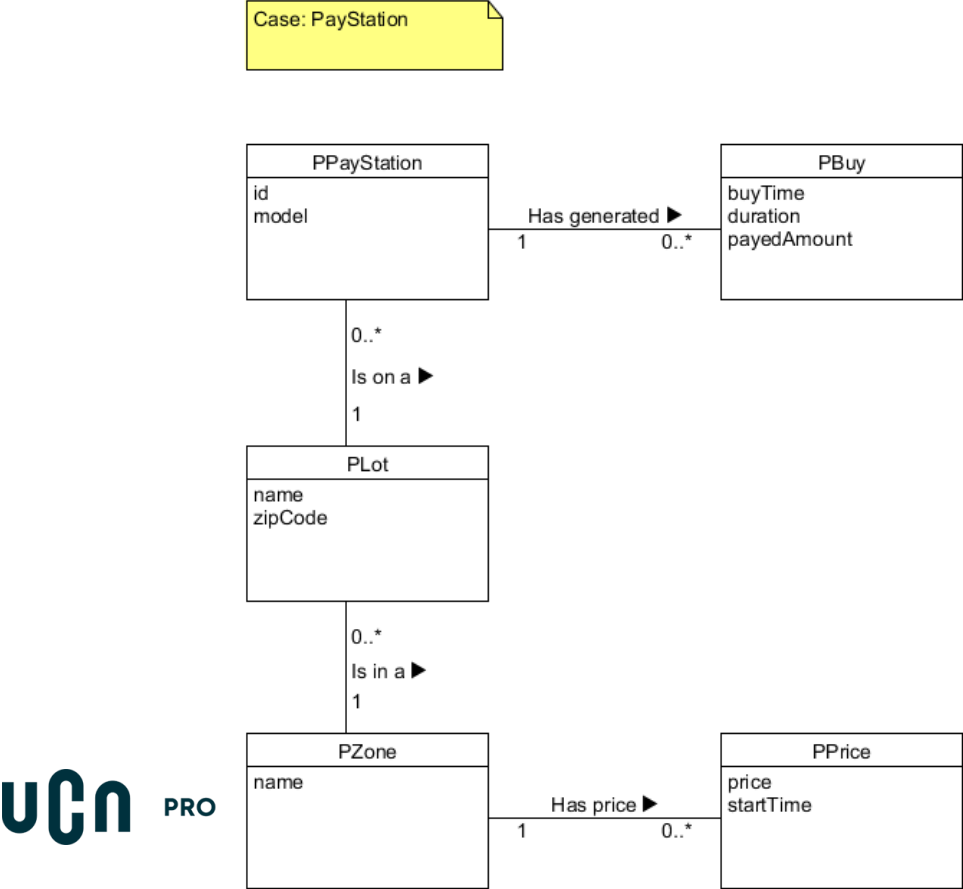


Unified Process Artifacts and Timing

Discipline	Artifact	Incep.	Elab.	Const.	Trans.
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	
Implementation (Code)	Implementation Model (code,....)		s	r	r

Example Pay Station

Domain Model and Relational Model



Example

Tables implemented in RDBMS

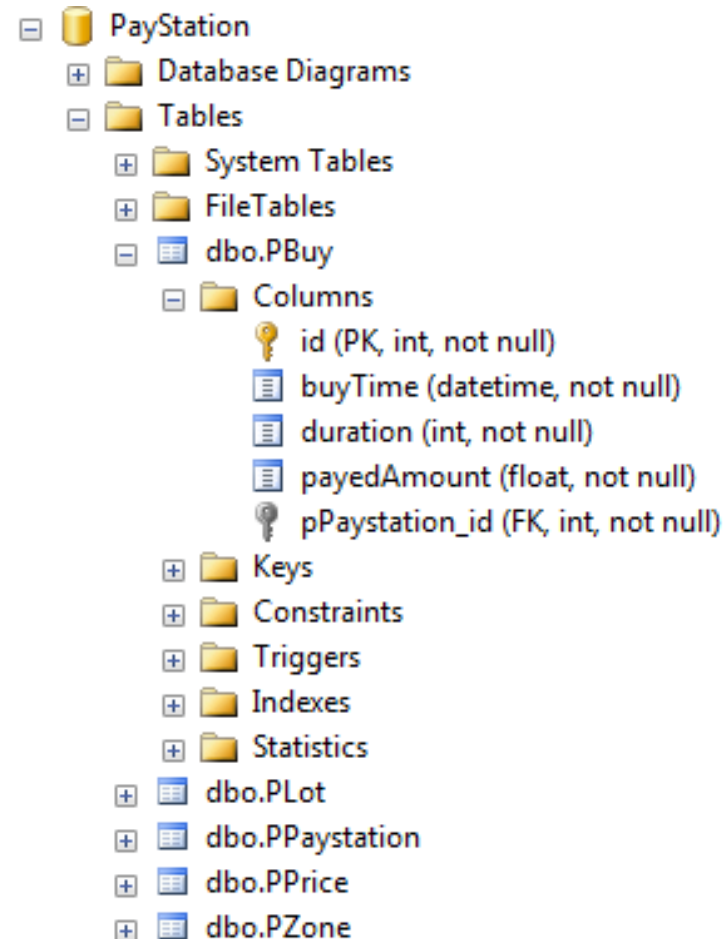
```
use PayStation;
/* First create PZone */
CREATE TABLE PZone
(
    id int IDENTITY(1,1) NOT NULL,
    name nvarchar(50) NOT NULL,
    CONSTRAINT PK_PZone PRIMARY KEY (id)
)

/* Create PPrice with FK to PZone */
CREATE TABLE PPrice
(
    id int IDENTITY(1,1) NOT NULL,
    price int NOT NULL,
    starttime datetime NOT NULL,
    pZone_id int NOT NULL,
    CONSTRAINT [PK_PPrice] PRIMARY KEY (id),
    CONSTRAINT FK_PPrice_PZone FOREIGN KEY(pZone_id) REFERENCES PZone(id)
)

/* Create PPlot with FK to PZone */
CREATE TABLE dbo.PPlot
(
    id int IDENTITY(1,1) NOT NULL,
    name nvarchar(50) NOT NULL,
    zipCode int NOT NULL,
    pZone_id int NOT NULL,
    CONSTRAINT PK_PPlot PRIMARY KEY (id),
    CONSTRAINT FK_PPlot_PZone FOREIGN KEY(pZone_id) REFERENCES PZone(id)
)

/* Create PPaystation with FK to PPlot */
CREATE TABLE dbo.PPaystation
(
    id int IDENTITY(1,1) NOT NULL,
    model nchar(10) NULL,
    pPlot_id int NOT NULL,
    CONSTRAINT PK_PPaystation PRIMARY KEY (id),
    CONSTRAINT FK_PPaystation_PPlot FOREIGN KEY(plot_id) REFERENCES Plot(id)
)

/* Create PBuy with FK to PPaystation */
CREATE TABLE dbo.PBuy(
    id int IDENTITY(1,1) NOT NULL,
    buyTime datetime NOT NULL,
    duration int NOT NULL,
    payedAmount float NOT NULL,
    pPaystation_id int NOT NULL,
    CONSTRAINT PK_PBuy PRIMARY KEY CLUSTERED (id),
    CONSTRAINT FK_PBuy_PPaystation FOREIGN KEY(pPaystation_id) REFERENCES PPaystation(id)
)
```



Example

From Domain Model to Relational Model (Database)

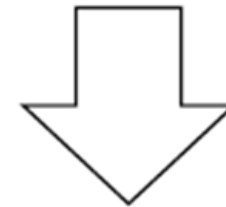
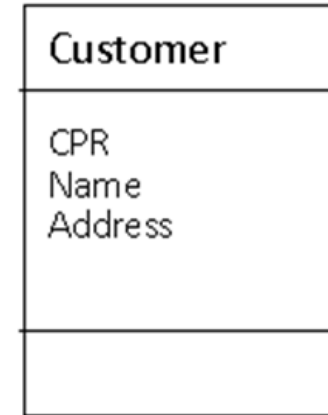


PROFESSIONSHØJSKOLEN

Map Domain Model → Relational Model

Each Conceptual Class is mapped to a table

- Class name is used as entity (table) name
 - Attributes are columns
 - A primary key is chosen or added
- For each attribute consider:
- Datatype
 - NULL values allowed? (the fewer the better)
 - Uniqueness – is it a key?



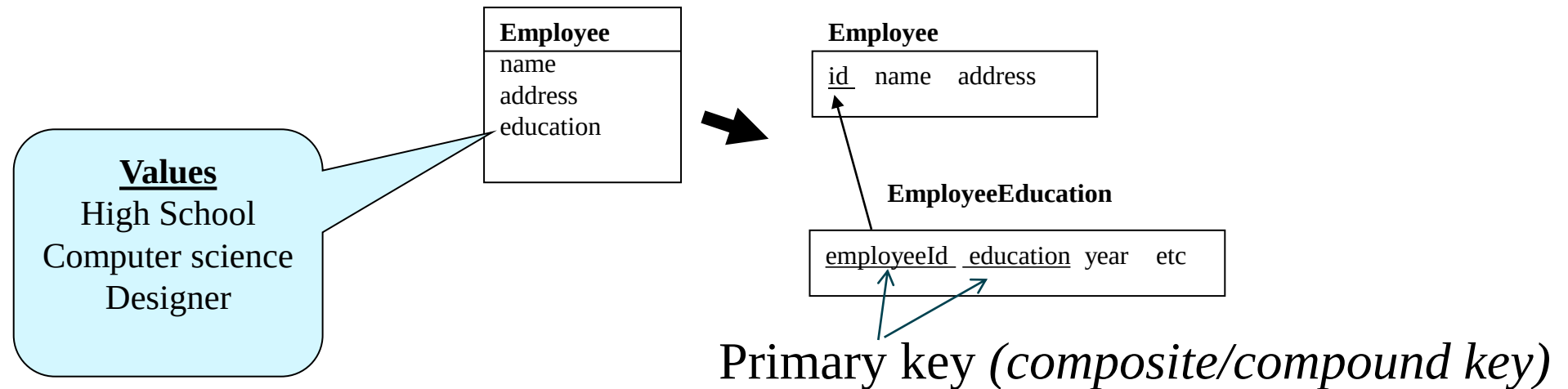
customer-ID	CPR	name	address
1	010155-2321	Jens Andersen	Søndergade 6
2	101289-7566	Oda Nielsen	Algade 99
•			
•			
•			
1251	060967-2390	Pia Schrøder	Bispensgade 27

Mapping rules

- **Multi-valued attributes not allowed**
 - Emails, children, telephone numbers, courses
- **Composite attributes are best represented by its components**
 - Split: Name -> first name, middle name, last name
 - Split: Address: -> street name, number, city...
- **Minimize null values**
 - Remember when choosing between transform alternatives

Multi-valued Attributes

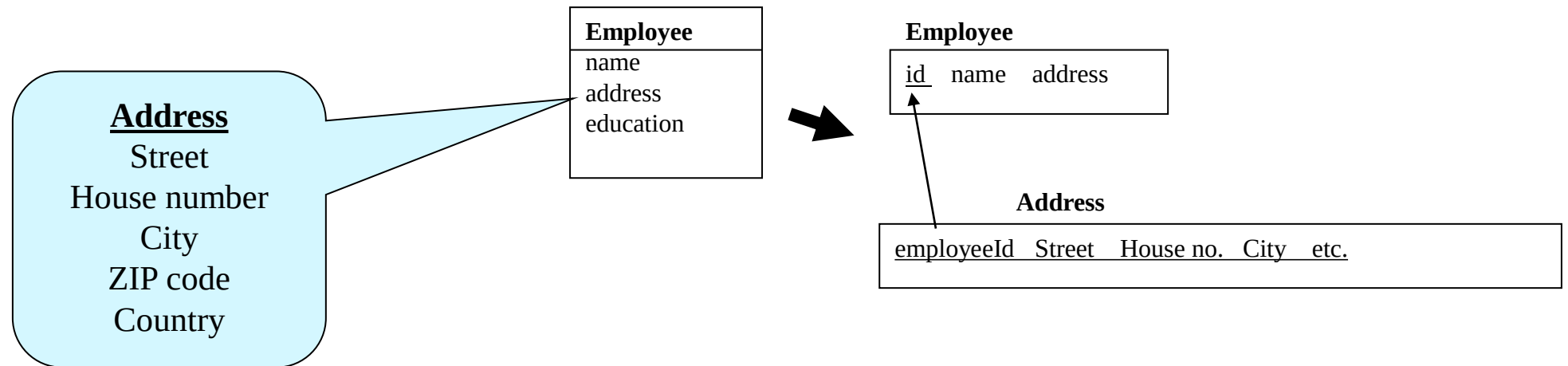
- For each multi-valued attribute, create a new table (entity), which includes the values of the attribute as well as the primary key from the original table which becomes a foreign key
- Example:



Consider: Change Domain Model to include Conceptual Class **Education**

Composite Attributes

- For any attribute that has child-attributes, create a new table (entity)
- Example:



Consider: Change Domain Model to include Conceptual Class **Address**

Sample Relational Database Schema

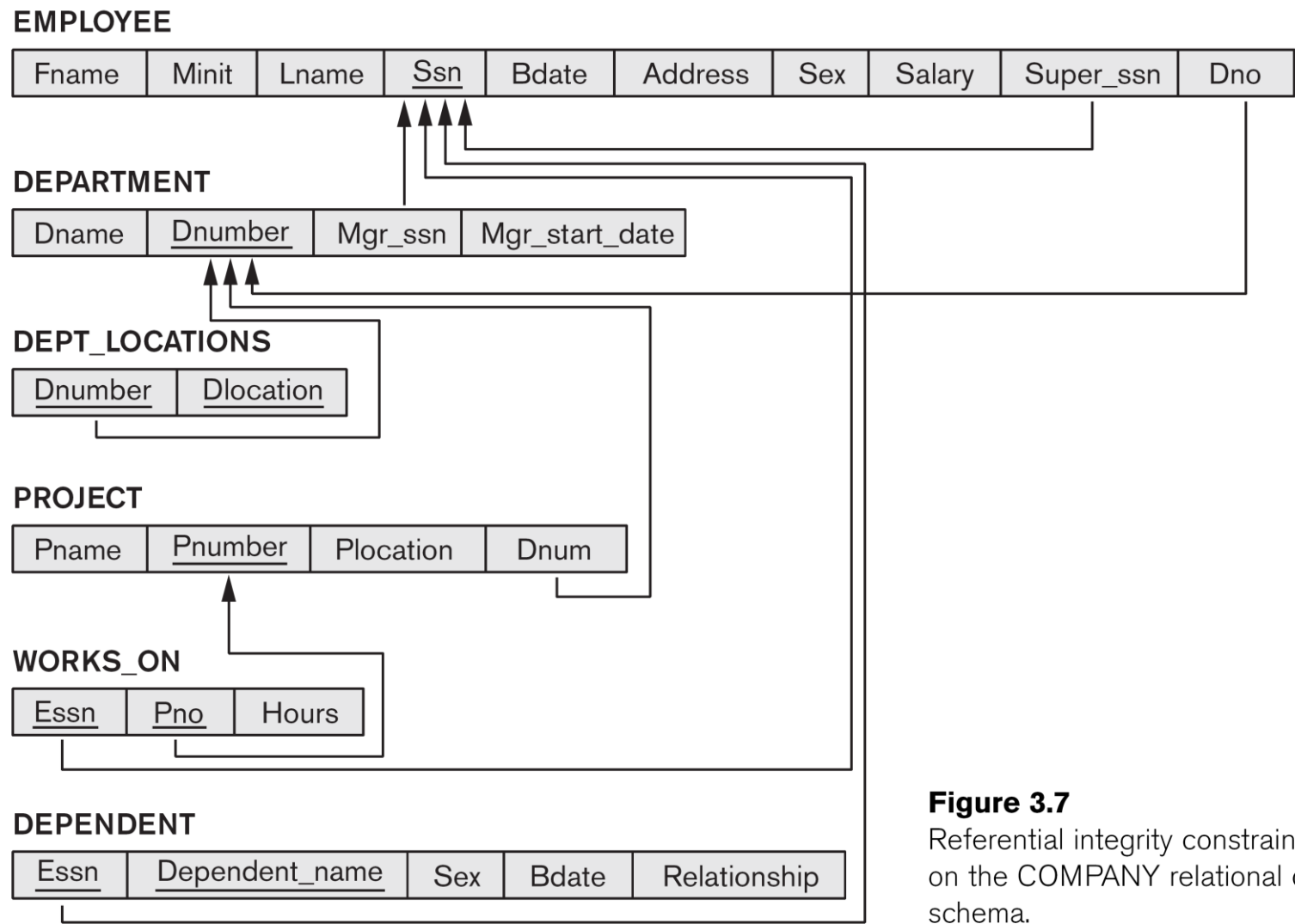
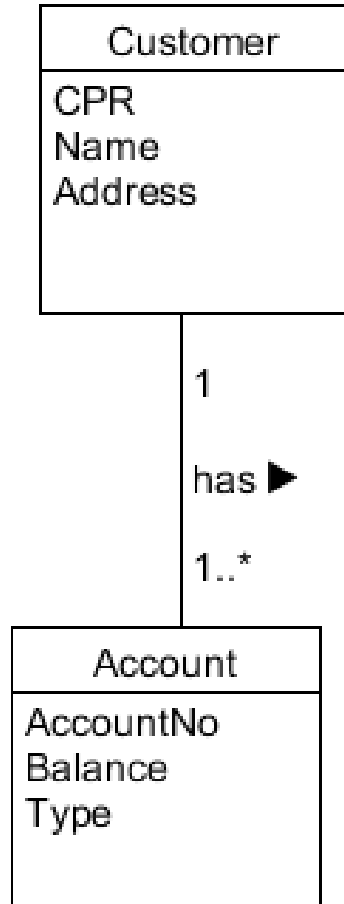


Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

source: [Elmasri]

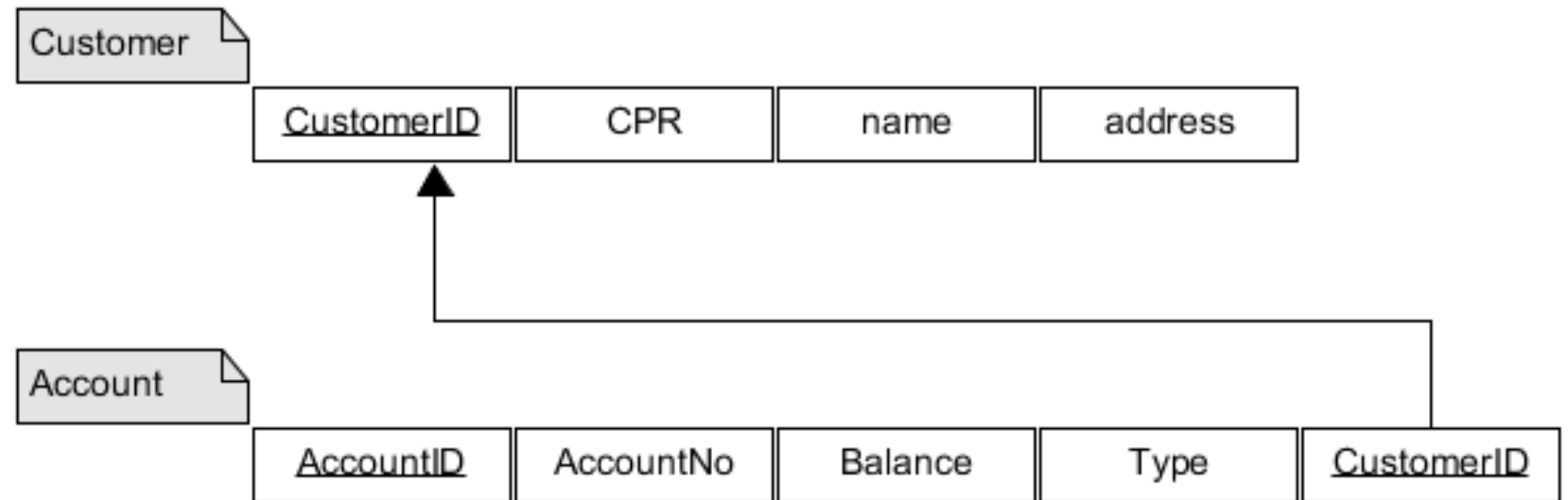
Mapping relations

One-to-many association



Two alternatives

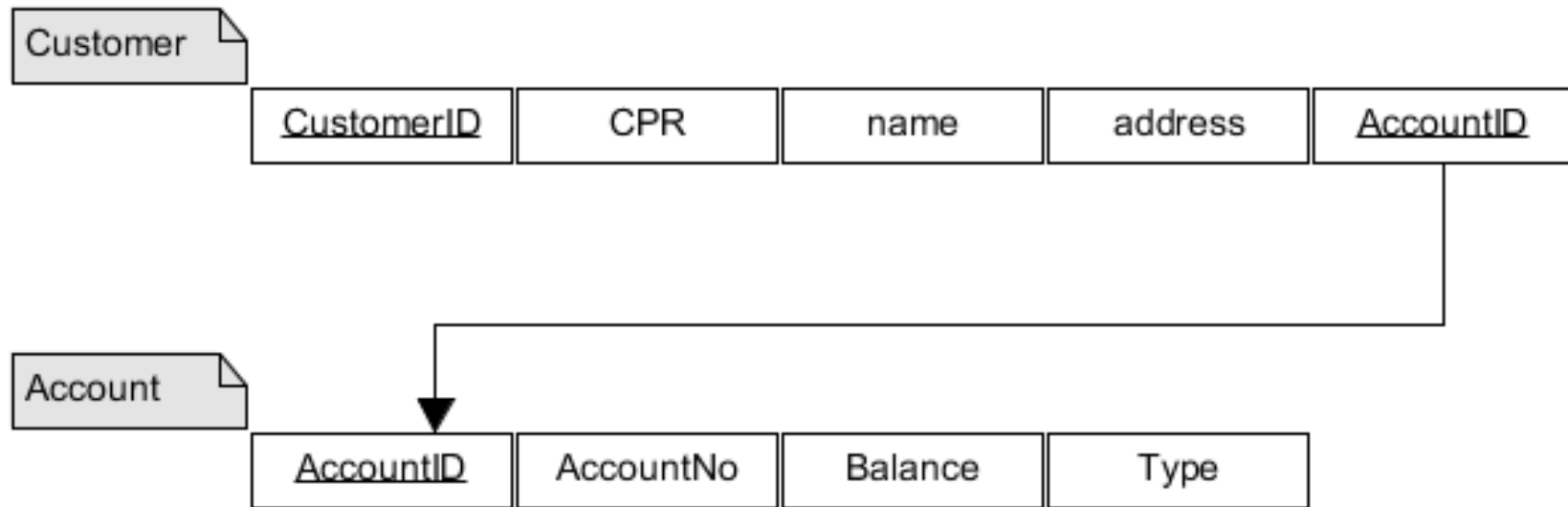
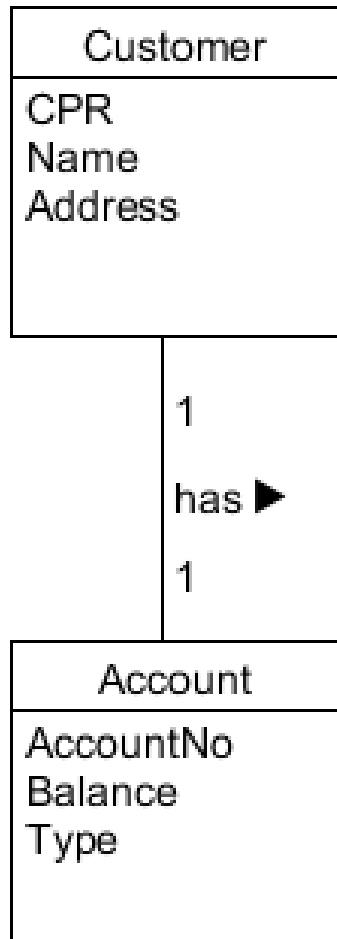
1. The key from the one-side is added as an attribute to the many-side as a foreign key (most often used)
2. Create a join table (as with many-to-many relationships)



One-to-one associations

Three alternatives

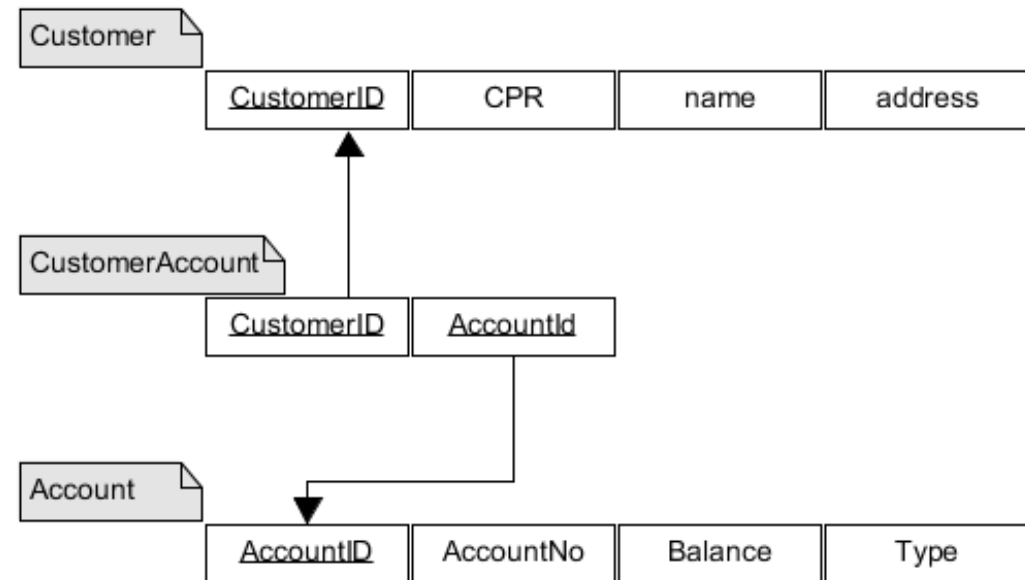
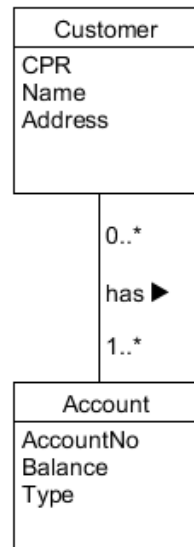
1. Two tables joined with foreign key (The key from one of the sides is added as an attribute to the other side as a foreign key. But at which side?)
2. Make one table with all attributes
3. Create a join table (*as with many-to-many relationships*)



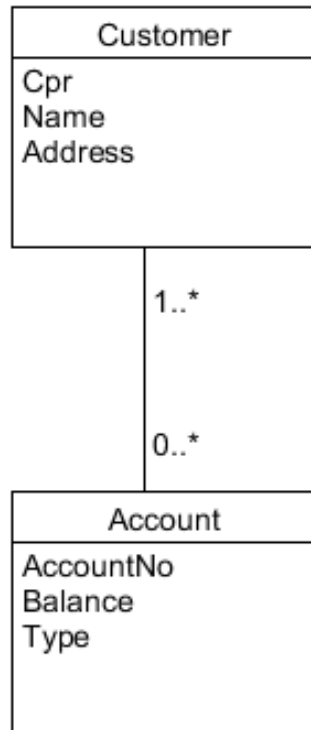
Is this the best solution?
...or should you have Account point to Customer?

Many-to-many association

- Create a new “join table”
 - Include the primary keys of the entities/tables that the “join table” joins as foreign key attributes (this combination will form the primary key of the “join table”). Include any simple attributes of the relationship type as attributes (time, created by, etc.)



Many-to-many association example



Customer

<u>CustomerID</u>	CPR	Name	Address
1	010155-2321	Jens Andersen	Søndergade 6
2	101289-7566	Oda Nielsen	Algade 99
•			
•			
•			
1251	060967-2390	Pia Schrøder	Bispensgade 27

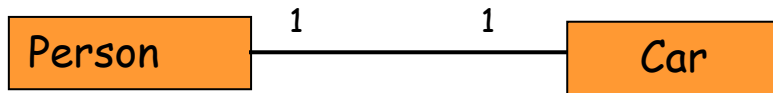
Account

<u>AccountId</u>	accountno	Balance	type
1	615-6789	280295	checkkonto
2	931-1453	311294	lån
•			
•			
•			
256	112-7290	120395	checkkonto

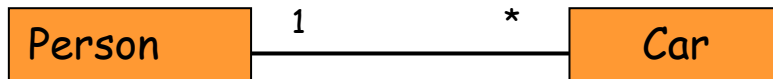
CustomerAccount

<u>CustomerId</u>	<u>AccountID</u>
1	2
1	4
2	1
2	2
2	4
3	3
4	2
4	1251
5	1251
•	
•	
•	
256	25

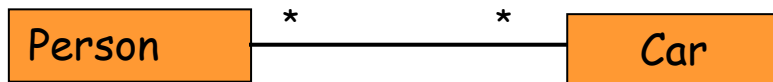
Associations - summary



- One-to-one (1-1):
 - Include the primary key from one of the sides on the other side as foreign key.



- One-to-many (1-*):
 - Include the primary key from the one-side on the many-side as foreign key



- Many-to-many (*-*):
 - Create a new “join table” with the primary keys from both sides as foreign keys. The combination of the two foreign keys becomes primary key in the “join table”.

Exercise 01

Domain model and relational model - Denmark

Draw a domain model for regions, provinces and municipalities in Denmark.
Map the domain model to a relational schema.

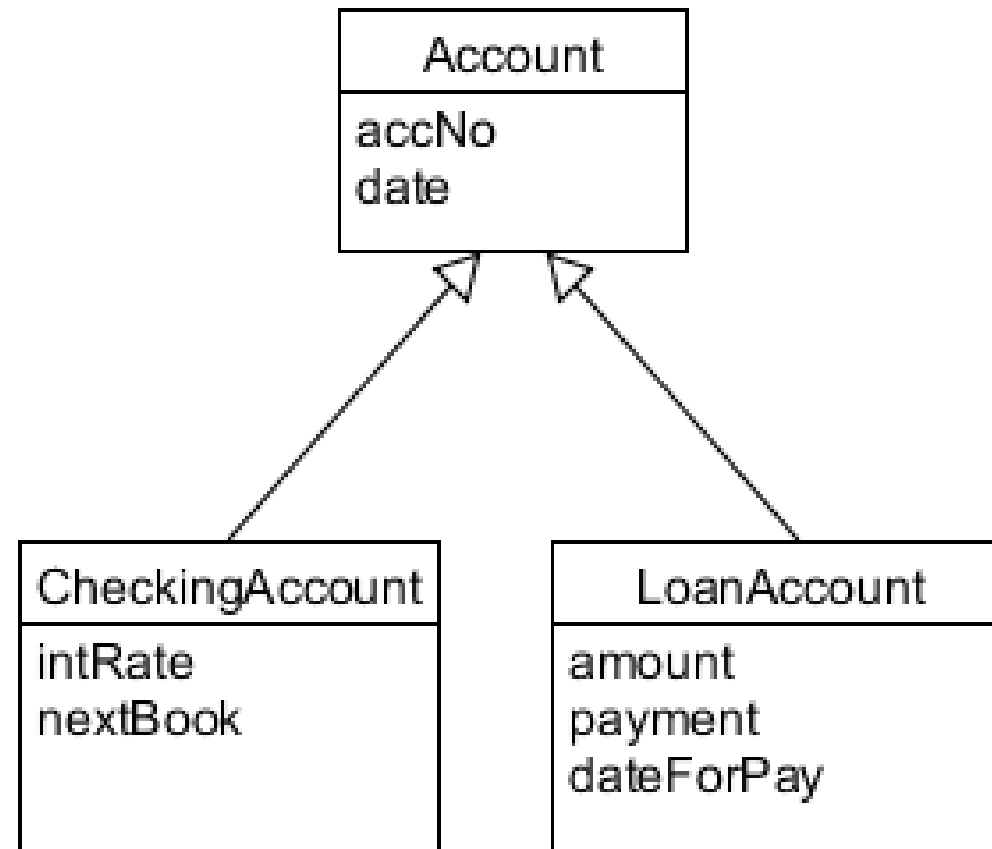
<https://www.dst.dk/en/Statistik/dokumentation/nomenklaturer/nuts>

Generalization

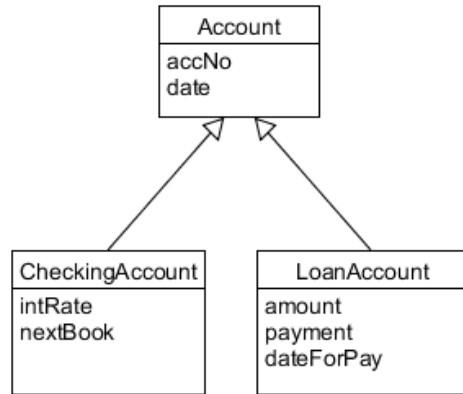
Generalization (class inheritance)

Three Approaches for Transformation

1. “Table-Each”
2. “Pull-Down”
3. “Pull-Up”



1. Generalization – Table-Each (Table for each class)



1. Each class is mapped into a table. The general and specific parts of an object are bonded together with keys.

Account

<u>accID</u>	accNo	date	accType
1	615-6789	280295	checkAcc
2	931-1453	311294	loan
•			
•			
•			
256	112-7290	120395	checkAcc

Add a type attribute

Primary keys in the subclasses are also foreign keys.

CheckingAccount

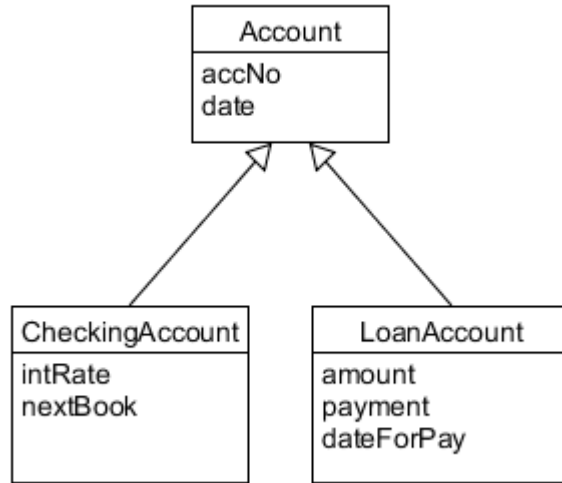
<u>accID</u>	intRate	bookNo
1	0,1	100395
•		
•		
•		
256	0,5	221294

LoanAccount

<u>accID</u>	amount	payment	dateForPay
2	25000	2500	30
•			
•			
•			

- Conceptually clear
The design can always be used.
- Update and reading difficult:
We must find data in two tables
- Accessing objects requires joins

2. Generalization - Pull Down



CheckingAccount

accD	accNo	date	intRate	nextBook
1	615-6789	280295	0,1	100395
⋮				
256	112-7290	120395	0,5	221294

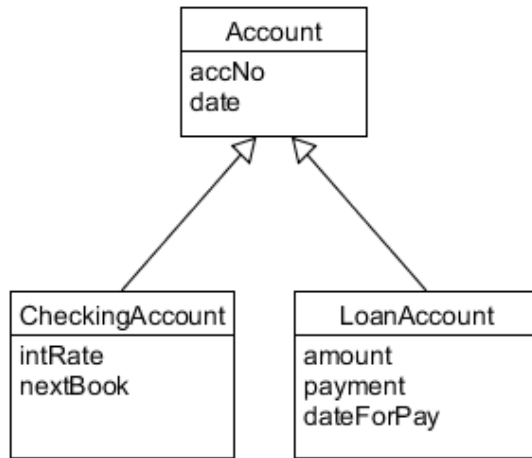
Loan

accD	accNo	date	amount	payment
2	615-6789	280295	25000	2500
⋮				
⋮				
⋮				

2. Each specialization class is mapped into a table that also contains the generalization of the class attributes. The attributes are “pulled down”.

- No table for the super-class. Changes in superclass, require update in both subclasses
- Works best, if there are few attributes in the super-class.
- Easy to retrieve all information if the type is known.
- Does not work if objects may belong to more than one subclass (multiple inheritance), or if objects of the superclass exists

3. Generalization - Pull Up



3. Generalization class mapped to a table that also contains all specialization classes attributes. The attributes are "pulled up".

- No tables for subclasses.
- Easy access.
- Works best if there are few attributes in the subclasses.
- Problems with null values

Account

accID	accNo	date	accType	intRate	nextBook	amount	payment	dateForPay
1	615-6789	280295	checkAcc	0,1	100395	NULL	NULL	NULL
2	931-1453	311294	loan	NULL	NULL	25000	2500	30
⋮								
256	112-7290	120395	checkAcc	0,5	221294	NULL	NULL	NULL

Summary (cheat sheet)

- **Each domain class is mapped to a table:**
 - Find or add **primary key** (single column, composite or surrogate key)
 - Turn **attributes** into columns (find types, determine if nulls are ok)
 - Split **multi-valued and composite attributes** into separate tables with foreign key
- **Associations and aggregations are represented by foreign key references:**
 - **1-1** Include the primary key of the one-side as foreign key on the other side.
 - **0-*** Include the primary key of the one-side as foreign key on the many-side.
 - ***-*** Create a join table with foreign keys pointing to primary keys of both tables
 - You can always use a join table. It is necessary if you want to store data on the relation.
- **Generalisation: choose one of these designs:**
 - **Table each** (one table for each class – avoids duplicate columns => joins for CRUD)
 - **Pull down** (no super class table, super class' attributes are in subclass tables => duplicate columns)
 - **Pull up** (only one table for the superclass, with columns for all attributes of subclasses => null values)

Exercise



PROFESSIONSHØJSKOLEN

Exercises

- Get as far as you can with "Module 05 Exercises"